

【技术分享】改进官方TF源码，进行BERT文本分类的多卡训练

 **腾讯云TI平台**
基于腾讯云强大计算能力的一站式机器学习生态服务平台

25 人赞同了该文章

本文原作者：彭浩源，经授权后发布。

原文链接：cloud.tencent.com/devel...

导语

Google-research开源的BERT代码中，微调BERT进行文本分类的demo代码是基于TPUEstimator的单卡实现，即使机器上有多块GPU，也无法并行训练，不满足大规模训练的要求。本文分析开源demo代码无法多卡训练的原因，并给出修改代码，支持多卡训练的方案。

1. 在多卡机器上单卡运行

Google-research开源的BERT官方Tensorflow代码（[项目地址](#)）中，提供了run_classifier.py作为接口，用于finetune一个文本分类模型。在run_classifier.py的代码中，使用tf.estimator进行模型的训练和评估，部分代码如下（run_classifier.py第847-880行）：

```
model_fn = model_fn_builder(  
    bert_config=bert_config,  
    num_labels=len(label_list),  
    init_checkpoint=FLAGS.init_checkpoint,  
    learning_rate=FLAGS.learning_rate,
```

▲ 赞同 25



● 3 条评论

🔗 分享

♥ 喜欢

★ 收藏

📄 申请转载





```
use_tpu=FLAGS.use_tpu,
use_one_hot_embeddings=FLAGS.use_tpu)

# If TPU is not available, this will fall back to normal Estimator on CPU
# or GPU.
estimator = tf.contrib.tpu.TPUEstimator(
    use_tpu=FLAGS.use_tpu,
    model_fn=model_fn,
    config=run_config,
    train_batch_size=FLAGS.train_batch_size,
    eval_batch_size=FLAGS.eval_batch_size,
    predict_batch_size=FLAGS.predict_batch_size)

if FLAGS.do_train:
    train_file = os.path.join(FLAGS.output_dir, "train.tf_record")
    file_based_convert_examples_to_features(
        train_examples, label_list, FLAGS.max_seq_length, tokenizer,
        tf.logging.info("***** Running training *****")
        tf.logging.info("  Num examples = %d", len(train_examples))
        tf.logging.info("  Batch size = %d", FLAGS.train_batch_size)
        tf.logging.info("  Num steps = %d", num_train_steps)
        train_input_fn = file_based_input_fn_builder(
            input_file=train_file,
            seq_length=FLAGS.max_seq_length,
            is_training=True,
            drop_remainder=True)
    estimator.train(input_fn=train_input_fn, max_steps=num_train_steps)
```

在这段代码中：

- 首先调用先前定义的model_fn_builder函数，这个函数的返回值也是一个函数，称为model_fn。这个model_fn定义了计算图的结构，以及mode分别为TRAIN, EVAL和PREDICT时的不同操作。

▲ 赞同 25 ▼

● 3 条评论

🔗 分享

♥ 喜欢

★ 收藏

📄 申请转载

...



tf.estimator.Estimator。

- 然后，调用file_based_input_fn_builder函数，这个函数的返回值也是一个函数，称为train_input_fn。这个train_input_fn的返回值是一个tf.data.Dataset。
- 最后，使用estimator.train()方法，传入train_input_fn和max_steps，开始训练。

这是使用estimator API进行模型训练的基本流程。使用这一流程进行训练有一个很大的问题：

即使机器上有多块GPU，在默认配置下，它只能使用一块GPU，无法充分利用GPU的算力。

在一台有8块P40的机器上，使用tensorflow1.15和python3运行run_classifier.py，在开始训练后，如果执行nvidia-smi命令查看GPU的使用情况，会得到这样的结果：

▲ 赞同 25 ▼

● 3 条评论

🔗 分享

♥ 喜欢

★ 收藏

📄 申请转载

...



```
Every 0.1s: nvidia-smi
Mon Dec 16 21:21:47 2019
```

NVIDIA-SMI 418.67				Driver Version: 418.67		CUDA Version: 10.1	
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr.	ECC
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage	GPU-Util	Compute	M.
0	Tesla P40	Off	00000000:04:00.0	Off	0		
N/A	35C	P0	151W / 250W	21921MiB / 22919MiB	100%	Default	
1	Tesla P40	Off	00000000:05:00.0	Off	0		
N/A	29C	P0	49W / 250W	161MiB / 22919MiB	0%	Default	
2	Tesla P40	Off	00000000:08:00.0	Off	0		
N/A	28C	P0	48W / 250W	161MiB / 22919MiB	0%	Default	
3	Tesla P40	Off	00000000:09:00.0	Off	0		
N/A	29C	P0	49W / 250W	161MiB / 22919MiB	0%	Default	
4	Tesla P40	Off	00000000:85:00.0	Off	0		
N/A	27C	P0	48W / 250W	161MiB / 22919MiB	0%	Default	
5	Tesla P40	Off	00000000:86:00.0	Off	0		
N/A	29C	P0	50W / 250W	161MiB / 22919MiB	0%	Default	
6	Tesla P40	Off	00000000:89:00.0	Off	0		
N/A	28C	P0	49W / 250W	161MiB / 22919MiB	0%	Default	
7	Tesla P40	Off	00000000:8A:00.0	Off	0		
N/A	28C	P0	49W / 250W	161MiB / 22919MiB	0%	Default	

Processes:				GPU Memory
GPU	PID	Type	Process name	Usage
0	49851	C	python	21907MiB
1	49851	C	python	147MiB
2	49851	C	python	147MiB
3	49851	C	python	147MiB
4	49851	C	python	147MiB
5	49851	C	python	147MiB
6	49851	C	python	147MiB
7	49851	C	python	147MiB

启动命令为:

▲ 赞同 25



● 3 条评论

🔗 分享

❤ 喜欢

★ 收藏

📄 申请转载

...

```
python run_classifier.py --data_dir /google-bert/CoLA --task_name cola --vocab_file /d
```



如图，机器上有8块GPU，但是只有GPU:0正在计算，其余GPU均处于闲置状态。

CoLA数据集共有8551个训练样本，我们使用的batch_size为默认值32，训练epoch数为3.0，因此总训练步数为 $8551 * 3 / 32 = 801$ 步。日志显示global_step/sec的值约为1.69，因此总训练时长约为474秒。

实验结果与预期相符，大约8分钟后，代码运行完毕，共训练801个step，验证集上的准确率为79.0%。

2. 直接加入MirroredStrategy (失败)

对于tf.estimator，常见的多卡分布式方案是使用tf.distribute.MirroredStrategy。我们尝试简单修改代码，在run_config中加入MirroredStrategy。修改后的代码如下：

```
# Add MirroredStrategy
strategy = tf.distribute.MirroredStrategy()
run_config = tf.contrib.tpu.RunConfig(
    cluster=tpu_cluster_resolver,
    master=FLAGS.master,
    model_dir=FLAGS.output_dir,
    save_checkpoints_steps=FLAGS.save_checkpoints_steps,
    train_distribute=strategy,
    tpu_config=tf.contrib.tpu.TPUConfig(
        iterations_per_loop=FLAGS.iterations_per_loop,
        num_shards=FLAGS.num_tpu_cores,
        per_host_input_for_training=is_per_host))
```

重新运行后，运行失败，Traceback显示：

▲ 赞同 25



● 3 条评论

🔗 分享

♥ 喜欢

★ 收藏

📄 申请转载





```

ERROR:tensorflow:Error recorded from training_loop: _call_input_fn() takes 3 positional arguments but 4 were given
E1217 02:53:53.220854 140219067856704 error_handling.py:75] Error recorded from training_loop: _call_input_fn() takes 3 positional arguments but 4 were given
INFO:tensorflow:training loop marked as finished
I1217 02:53:53.220967 140219067856704 error_handling.py:101] training_loop marked as finished
WARNING:tensorflow:Reraising captured error
W1217 02:53:53.221059 140219067856704 error_handling.py:135] Reraising captured error
Traceback (most recent call last):
  File "run_classifier_distribute.py", line 993, in <module>
    tf.app.run()
  File "/usr/local/lib/python3.6/dist-packages/tensorflow_core/python/platform/app.py", line 40, in run
    _run(main=main, argv=argv, flags_parser=_parse_flags_tolerate_undef)
  File "/usr/local/lib/python3.6/dist-packages/absl/app.py", line 299, in run
    run_main(main, args)
  File "/usr/local/lib/python3.6/dist-packages/absl/app.py", line 250, in _run_main
    sys.exit(main(argv))
  File "run_classifier_distribute.py", line 890, in main
    estimator.train(input_fn=train_input_fn, max_steps=num_train_steps)
  File "/usr/local/lib/python3.6/dist-packages/tensorflow_estimator/python/estimator/tpu/tpu_estimator.py", line 3035, in train
    rendezvous.raise_errors()
  File "/usr/local/lib/python3.6/dist-packages/tensorflow_estimator/python/estimator/tpu/error_handling.py", line 136, in raise_errors
    six.reraise(typ, value, traceback)
  File "/usr/lib/python3/dist-packages/six.py", line 693, in reraise
    raise value
  File "/usr/local/lib/python3.6/dist-packages/tensorflow_estimator/python/estimator/tpu/tpu_estimator.py", line 3030, in train
    saving_listeners=saving_listeners)
  File "/usr/local/lib/python3.6/dist-packages/tensorflow_estimator/python/estimator/estimator.py", line 370, in train
    loss = self._train_model(input_fn, hooks, saving_listeners)
  File "/usr/local/lib/python3.6/dist-packages/tensorflow_estimator/python/estimator/estimator.py", line 1159, in _train_model
    return self._train_model_distributed(input_fn, hooks, saving_listeners)
  File "/usr/local/lib/python3.6/dist-packages/tensorflow_estimator/python/estimator/estimator.py", line 1222, in _train_model_distributed
    self._config_train_distribute, input_fn, hooks, saving_listeners)
  File "/usr/local/lib/python3.6/dist-packages/tensorflow_estimator/python/estimator/estimator.py", line 1258, in _actual_train_model_distributed
    input_fn, ModeKeys.TRAIN, strategy)
  File "/usr/local/lib/python3.6/dist-packages/tensorflow_estimator/python/estimator/estimator.py", line 1012, in _get_iterator_from_input_fn
    lambda input_context: self._call_input_fn(input_fn, mode,
  File "/usr/local/lib/python3.6/dist-packages/tensorflow_core/python/distribute/distribute_lib.py", line 1050, in make_input_fn_iterator
    input_fn, replication_mode)
  File "/usr/local/lib/python3.6/dist-packages/tensorflow_core/python/distribute/distribute_lib.py", line 577, in make_input_fn_iterator
    input_fn, replication_mode=replication_mode)
  File "/usr/local/lib/python3.6/dist-packages/tensorflow_core/python/distribute/mirrored_strategy.py", line 552, in _make_input_fn_iterator
    self._container_strategy())
  File "/usr/local/lib/python3.6/dist-packages/tensorflow_core/python/distribute/input_lib.py", line 719, in _init_
    result = input_fn(ctx)
  File "/usr/local/lib/python3.6/dist-packages/tensorflow_estimator/python/estimator/estimator.py", line 1013, in <lambda>
    input_context))
TypeError: _call_input_fn() takes 3 positional arguments but 4 were given

```

Traceback中的信息比较有限，难以排查具体错误原因，但我们可以初步得出结论，无法直接将MirroredStrategy应用到TPUEstimator中。

3. 改用普通Estimator和MirroredStrategy (失败)

由于我们是在GPU机器上训练，不使用TPU，因此我们尝试将TPUEstimator改为普通tf.estimator.Estimator。

首先我们修改run_config，将run_classifier.py中828行定义的run_config改为：

```

# Add MirroredStrategy
strategy = tf.distribute.MirroredStrategy()
run_config = tf.estimator.RunConfig(
    model_dir=FLAGS.output_dir,

```

▲ 赞同 25



● 3 条评论

🔗 分享

♥ 喜欢

★ 收藏

📄 申请转载

...



其次，修改estimator，将run_classifier.py中859行定义的estimator改为：

```
estimator = tf.estimator.Estimator(model_fn=model_fn, config=run_config)
```

然后，我们修改file_based_input_fn_builder函数。由于原有的file_based_input_fn_builder中返回的input_fn函数签名中包含参数params，并从params中读取batch_size，但我们定义的普通estimator的默认params中并没有这一参数，因此，我们将【从params中读取batch_size】改为【将batch_size作为参数传入file_based_input_fn_builder】。修改后的file_based_input_fn_builder函数如下：

```
def file_based_input_fn_builder(input_file, seq_length, is_training,
                                drop_remainder, batch_size):
    """Creates an `input_fn` closure to be passed to TPUEstimator."""

    name_to_features = {
        "input_ids": tf.FixedLenFeature([seq_length], tf.int64),
        "input_mask": tf.FixedLenFeature([seq_length], tf.int64),
        "segment_ids": tf.FixedLenFeature([seq_length], tf.int64),
        "label_ids": tf.FixedLenFeature([], tf.int64),
        "is_real_example": tf.FixedLenFeature([], tf.int64),
    }

    def _decode_record(record, name_to_features):
        """Decodes a record to a TensorFlow example."""
        example = tf.parse_single_example(record, name_to_features)

        # tf.Example only supports tf.int64, but the TPU only supports tf.int32.
        # So cast all int64 to int32.
        for name in list(example.keys()):
            t = example[name]
            if t.dtype == tf.int64:
                t = tf.to_int32(t)
```

▲ 赞同 25



● 3 条评论

🔗 分享

♥ 喜欢

★ 收藏

📄 申请转载





```
return example

def input_fn():
    """The actual input function."""
    # For training, we want a lot of parallel reading and shuffling.
    # For eval, we want no shuffling and parallel reading doesn't matter.
    d = tf.data.TFRecordDataset(input_file)
    if is_training:
        d = d.repeat()
        d = d.shuffle(buffer_size=100)

    d = d.apply(
        tf.contrib.data.map_and_batch(
            lambda record: _decode_record(record, name_to_features),
            batch_size=batch_size,
            drop_remainder=drop_remainder))

    return d

return input_fn
```

在调用时，我们需要多传一个参数batch_size:

```
train_input_fn = file_based_input_fn_builder(
    input_file=train_file,
    seq_length=FLAGS.max_seq_length,
    is_training=True,
    drop_remainder=True,
    batch_size=FLAGS.train_batch_size)
```

生成eval_input_fn和predict_input_fn时，应分别传入对应的batch_size。

最后，我们要修改model_fn本身，将不同mode下的返回值类型从
tf.contrib.train.EstimatorSpec改为tf.estimator.EstimatorSpec 对于train和predict两个

▲ 赞同 25



● 3 条评论

🔗 分享

❤ 喜欢

★ 收藏

📄 申请转载

...

mode, 直接修改返回值类型即可, 对于eval这个mode, 我们需要改变eval_metrics的类型, 修改后的具体代码如下, 即将eval_metrics变为了eval_metric_ops:

```
eval_metric_ops = metric_fn(per_example_loss, label_ids,
                             logits, is_real_example)
output_spec = tf.estimator.EstimatorSpec(
    mode=mode,
    loss=total_loss,
    eval_metric_ops=eval_metric_ops)
```

完成以上修改后, 运行代码, 出现了一个ValueError, 内容如下:

```
ValueError: You must specify an aggregation method to update a MirroredVariable in Rep
```

观察Traceback, 可以发现, 这个ValueError是optimizer在apply_gradients时产生的, 具体错误位于optimization.py的154行。

```
Traceback (most recent call last):
  File "/usr/local/lib/python3.6/dist-packages/tensorflow_core/python/training/coordinator.py", line 297, in stop_on_exception
    yield
  File "/usr/local/lib/python3.6/dist-packages/tensorflow_core/python/distribute/mirrored_strategy.py", line 880, in run
    self.main_result = self.main_fn(*self.main_args, **self.main_kwargs)
  File "/usr/local/lib/python3.6/dist-packages/tensorflow_estimator/python/estimator/estimator.py", line 1149, in _call_model_fn
    model_fn_results = self._model_fn(features=features, **kwargs)
  File "run_classifier_distribute.py", line 673, in model_fn
    total_loss, learning_rate, num_train_steps, num_warmup_steps, use_tpu)
  File "/google-bert/bert/optimization.py", line 77, in create_optimizer
    zip(grads, tvars), global_step=global_step)
  File "/google-bert/bert/optimization.py", line 154, in apply_gradients
    [param.assign(next_param),
  File "/usr/local/lib/python3.6/dist-packages/tensorflow_core/python/distribute/values.py", line 1036, in assign
    return self._assign_func(f=assign_fn, *args, **kwargs)
  File "/usr/local/lib/python3.6/dist-packages/tensorflow_core/python/distribute/values.py", line 1016, in _assign_func
    variable_type="MirroredVariable"))
```

在Google公开的BERT代码中, 从optimization.py可以看出, 模型训练时没有用tensorflow内置的优化器, 而是通过继承 `tf.train.Optimizer`, 并重写`apply_gradients`等方法自行实现了一个AdamWeightDecayOptimizer类作为优化器。Google-research的源代码中, 实现优化器时没有考虑到优化器和分布式训练的兼容, 没有定义优化器中的变量在多卡训练时的聚合(Aggregation)方式, 因而在多卡训练时会报错。

▲ 赞同 25



● 3 条评论

🔗 分享

♥ 喜欢

★ 收藏

📄 申请转载





4. 改用普通Estimator和MirroredStrategy，同时更换优化器（成功）

最简单的更换优化器的方式就是修改optimization.py。在optimization.py第60行中定义optimizer变量为同一个py文件中自行实现的AdamWeightDecayOptimizer，我们先尝试将其替换为普通的AdamOptimizer：

```
optimizer = tf.train.AdamOptimizer(  
    learning_rate=learning_rate,  
    epsilon=1e-6)
```

注意，由于Google-research实现的AdamWeightDecayOptimizer在apply_gradient方法中没有调整global_step，他们在optimization.py的第82-83行手动增加了global_step。如果我们换成普通的optimizer，需要删除这两行的代码，否则global_step的值会变成实际值的两倍！

重新运行后，运行成功，nvidia-smi命令显示，机器上的8张GPU全部处于运作状态，这也进一步验证了原有代码运行失败的原因出在优化器上。

▲ 赞同 25 ▼

● 3 条评论

🔗 分享

♥ 喜欢

★ 收藏

📄 申请转载

...



```
Every 0.4s: nvidia-smi
```

Tue Dec 17 14:42:24 2019

NVIDIA-SMI 418.67				Driver Version: 418.67		CUDA Version: 10.1	
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr. ECC	
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage	GPU-Util	Compute M.	
0	Tesla P40	Off	00000000:04:00.0	Off		0	
N/A	40C	P0	169W / 250W	21927MiB / 22919MiB	92%	Default	
1	Tesla P40	Off	00000000:05:00.0	Off		0	
N/A	40C	P0	175W / 250W	21921MiB / 22919MiB	92%	Default	
2	Tesla P40	Off	00000000:08:00.0	Off		0	
N/A	39C	P0	160W / 250W	21927MiB / 22919MiB	92%	Default	
3	Tesla P40	Off	00000000:09:00.0	Off		0	
N/A	40C	P0	164W / 250W	21927MiB / 22919MiB	92%	Default	
4	Tesla P40	Off	00000000:85:00.0	Off		0	
N/A	38C	P0	162W / 250W	21921MiB / 22919MiB	93%	Default	
5	Tesla P40	Off	00000000:86:00.0	Off		0	
N/A	40C	P0	165W / 250W	21927MiB / 22919MiB	92%	Default	
6	Tesla P40	Off	00000000:89:00.0	Off		0	
N/A	39C	P0	164W / 250W	21927MiB / 22919MiB	98%	Default	
7	Tesla P40	Off	00000000:8A:00.0	Off		0	
N/A	40C	P0	173W / 250W	21927MiB / 22919MiB	98%	Default	

Processes:				GPU Memory Usage
GPU	PID	Type	Process name	
0	28288	C	python	21913MiB
1	28288	C	python	21907MiB
2	28288	C	python	21913MiB
3	28288	C	python	21913MiB
4	28288	C	python	21907MiB
5	28288	C	python	21913MiB
6	28288	C	python	21913MiB
7	28288	C	python	21913MiB

但是，使用tensorflow内置的AdamOptimizer代替AdamWeightDecayOptimizer的话，就失去

▲ 赞同 25



● 3 条评论

🔗 分享

♥ 喜欢

★ 收藏

📄 申请转载

...



支持多卡的实现，如JayYip的实现，我们可以用这些多卡版本的AdamWeightDecayOptimizer代替BERT源码中的AdamWeightDecayOptimizer，从而满足对多卡训练的要求。

5. 根据GPU数量调整训练步数

在Google-research提供的源代码中是通过num_epochs控制训练步数的，如run_classifier.py第842-845行所示，代码中根据训练集样本个数，训练epoch数和训练时的batch_size计算出训练步数，具体的计算方式是： $\text{训练步数} = \text{训练集样本个数} * \text{训练epoch数} / \text{训练时batch_size}$ 。训练步数被用于控制estimator的训练。

当我们成功使用多卡并行训练时，每张卡上均有一个batch在训练，因此实际的batch_size是设定的batch_size与GPU数目的乘积，我们可以通过以下实验来简单证明这一点：

我们将file_based_input_fn_builder中 `d = d.repeat()` 一行去掉，同时将main函数中的 `estimator.train(input_fn=train_input_fn, max_steps=num_train_steps)` 改为 `estimator.train(input_fn=train_input_fn)`。这一步的目的是让dataset只有一份训练集的样本（即8551个训练样本各出现一次），且以遍历完dataset来作为训练结束的条件。同时，我们在启动命令中加入 `-train_batch_size 1`，将batch_size改为1。

完成上述改动后，我们使用8卡并行训练。我们发现，训练结束时global_step的值是1068，正好是 $8551 / 8$ ，这说明我们的训练策略是每张卡上的batch_size为1，由于有8张卡，实际每个step训练了8个batch。

因此，当多卡并行训练时，如果我们还是以epoch来控制训练步数，那么计算实际的max_steps时，要除以GPU的数量。

我们使用8块GPU，32的train_batch_size，在CoLA数据集上训练3.0个epoch，则实际的训练步数是 $8551 * 3 / 8 / 32 = 100$ 步。经过了3分32秒的准备时间和1分50秒的实际训练时间后，训练成功完成，验证集准确率与单卡训练相当，训练时间减少了约3分钟。这是由于多卡训练较长的准备时间导致的。CoLA数据集上的实验表明，8卡并行训练时，global_step/sec的值约为0.91，相比于单卡的1.69，训练速度是单卡的 $0.91 * 8 / 1.69 = 4.31$ 倍。因此当训练数据量很大时，多卡训练能显著减少训练时间。

▲ 赞同 25



● 3 条评论

🔗 分享

♥ 喜欢

★ 收藏

📄 申请转载





6. 其它注意事项

使用上述改动进行多卡训练时，要注意：

- 多卡并行的调度和同步等操作本身需要一定的时间。以前面CoLA数据集的实验为例，当使用8块P40GPU并行训练时，在执行训练命令大约3-4分钟后，实际的训练才开始。因此，是否使用多卡并行训练需要考虑训练量的大小。如果训练量不大的话，可能单卡训练的实际耗时反而更短。

7. 总结

综上所述，改动BERT的run_classifier.py以进行多卡训练需要以下步骤：

1. 将tf.contrib.tpu.TPUEstimator改为tf.estimator.Estimator。此处需要修改的代码较多，包括model_fn_builder和file_based_input_fn_builder，以及file_based_input_fn_builder的调用代码。
2. 在run_config中加入MirroredStrategy。
3. 修改optimization.py中的优化器，使用tensorflow内置的优化器或者支持多卡训练的AdamWeightDecayOptimizer的实现，此处需要注意优化器的apply_gradients方法是否增加了global_step，以决定是否需要手动增加global_step。
4. 如果按照epoch数控制训练，需要根据GPU的数量，调整传给estimator.train的max_steps参数。

发布于 2020-04-08 15:27

文本分类

BERT

3 条评论

切换为时间排序

赞同 25



3 条评论

分享

喜欢

收藏

申请转载





知乎用户

03-17

大哥，直接用官方文档里那个keras的bert不好么...

👍 赞



大导弹

2021-08-05

写的真清楚，想我这种外行都看得明明白白的，多谢👍

👍 赞



知乎用户

2020-09-11

您写的很详细很棒，想请问一下把AdamWeightDecayOptimizer换成AdamOptimizer的话会对finetune结果造成怎样的影响呢？

👍 赞

推荐阅读

bert中文微调

最近学习了一点儿关于bert中文微调的知识，并上手实践了一下，再次进行记录。bert中文微调需要三个“原料”首先是github上bert的源码，可以在bert源码下载，其次是Google预训练好的bert模...

许琛



加入Transformer-XL，这个PyTorch包能调用各种NLP预...

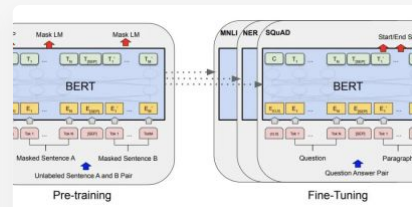
机器之心

发表于机器之心



干货 | BERT fine-tune 终极实践教程

Naturali 奇点机智



保姆级教程，用PyTorch和BERT进行文本分类

机器学习社...

发表于机器学习社...

▲ 赞同 25



● 3 条评论

🔗 分享

❤️ 喜欢

★ 收藏

📄 申请转载

